

Stroke Prediction — Data Cleaning & Preprocessing

ITAI 1371 Midterm — Team ML_1371_12321 (Edwin, Kala, Anashrah, Zain)

Dataset: Stroke Prediction Dataset (Kaggle, fedesoriano) — 5,110 rows × 12 columns
<https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset>

Goal of this notebook: take the raw dataset and produce a clean, balanced training set ready for model building, while keeping the test set untouched so our future evaluation is honest.

The golden rule we follow throughout:

Split FIRST → do everything else on training only → SMOTE-balance LAST. The test set is set aside immediately after the split and never sees imputation values, scaler ranges, or SMOTE derived from itself. Touching the test set corrupts the experiment.

Pipeline order (this order matters):

1. Load data, drop the `id` column
2. **Stratified 70/30 split** (test set set aside now)
3. **EDA on training data only** (Mr. Rao's requirement)
4. Impute missing `bmi` (median) — fit on train
5. One-hot encode categoricals + scale numerics — fit on train
6. **SMOTE on the training set only** — before/after plot
7. Save the clean balanced dataset

0. Setup

`SMOTE` lives in the `imbalanced-learn` library, which is **not** part of base scikit-learn, so we install it first. On Google Colab this cell needs to run once per session.

```
1 # Install imbalanced-learn (provides SMOTE). Safe to re-run.
2 !pip install imbalanced-learn -q
```

```
1 from google.colab import files
2 uploaded = files.upload()
```

`Choose Files` No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving healthcare-dataset-stroke-data.csv to healthcare-dataset-stroke-data.csv
Saving stroke_clean_train_balanced.csv to stroke_clean_train_balanced.csv
Saving stroke_processed_test.csv to stroke_processed_test.csv

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3
4 # 1. Load the pre-balanced training set your team generated
5 train_df = pd.read_csv('stroke_clean_train_balanced.csv')
6 X_train_balanced = train_df.drop(columns=['stroke'])
7 y_train_balanced = train_df['stroke']
8
9 # 2. Load the processed, clean evaluation set
10 test_df = pd.read_csv('stroke_processed_test.csv')
11 X_eval_raw = test_df.drop(columns=['stroke'])
12 y_eval_raw = test_df['stroke']
13
14 # 3. Partition into identical 15% Validation and 15% Test blocks
15 RANDOM_STATE = 42
16 X_val_processed, X_test_processed, y_val, y_test = train_test_split(
17     X_eval_raw, y_eval_raw, test_size=0.50, random_state=RANDOM_STATE, stratify=y_eval_raw
18 )
19
20 print(f"✅ Variables loaded successfully from your Shared Drive files!")
```

✅ Variables loaded successfully from your Shared Drive files!

```
1 # Core libraries
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5
6 # scikit-learn preprocessing tools
7 from sklearn.model_selection import train_test_split
8 from sklearn.impute import SimpleImputer
9 from sklearn.preprocessing import StandardScaler, OneHotEncoder
10 from sklearn.compose import ColumnTransformer
11 from sklearn.pipeline import Pipeline
12
13 # SMOTE for class balancing (training data only)
14 from imblearn.over_sampling import SMOTE
15
16 RANDOM_STATE = 42 # fixed seed so results are reproducible for everyone on the team
17 pd.set_option("display.max_columns", None)
18 print("Libraries loaded.")
```

Libraries loaded.

1. Load the raw data

The raw CSV records missing BMI values as the string `"N/A"`, so we tell pandas to read those as real missing values (`NaN`). We also drop `id`, which is just a row number and carries no predictive information.

```
1 # >>> If your file is named differently or lives in a different folder, change this line only. <<<
2 CSV_PATH = "healthcare-dataset-stroke-data.csv"
3
4 # na_values="N/A" makes pandas treat the literal string "N/A" as a missing value
5 df = pd.read_csv(CSV_PATH, na_values="N/A")
6
7 print("Raw shape:", df.shape) # expected: (5110, 12)
8 df.head()
```

Raw shape: (5110, 12)												
	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

```
1 # Structure and data types of every column
2 df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5110 entries, 0 to 5109
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   id                    5110 non-null   int64
1   gender                5110 non-null   object
2   age                   5110 non-null   float64
3   hypertension          5110 non-null   int64
4   heart_disease         5110 non-null   int64
5   ever_married          5110 non-null   object
6   work_type             5110 non-null   object
7   Residence_type        5110 non-null   object
8   avg_glucose_level     5110 non-null   float64
9   bmi                   4909 non-null   float64
10  smoking_status        5110 non-null   object
11  stroke                5110 non-null   int64
dtypes: float64(3), int64(4), object(5)
memory usage: 479.2+ KB
```

```
1 # Drop the id column – it's just an identifier, not a feature
2 df = df.drop(columns=["id"])
3 print("Shape after dropping id:", df.shape)
4 df.head()

Shape after dropping id: (5110, 11)
   gender  age  hypertension  heart_disease  ever_married  work_type  Residence_type  avg_glucose_level  bmi  smoking_status  stroke
0   Male  67.0             0             1           Yes    Private          Urban             228.69    36.6  formerly smoked      1
1  Female  61.0             0             0           Yes  Self-employed          Rural             202.21   NaN   never smoked      1
2   Male  80.0             0             1           Yes    Private          Rural             105.92   32.5   never smoked      1
3  Female  49.0             0             0           Yes    Private          Urban             171.23   34.4         smokes      1
4  Female  79.0             1             0           Yes  Self-employed          Rural             174.12   24.0   never smoked      1
```

2. Split FIRST — 70% train / 30% test (stratified)

This is the most important step for avoiding data leakage. We split **before** any cleaning, imputation, scaling, or balancing.

- `stratify=y` keeps the real stroke ratio (~5%) in *both* the training and test sets, so the test set still looks like the real world.
- After this cell, `X_test` / `y_test` are **set aside and never touched again** until the finals.

```
1 # Separate features (X) from the target we want to predict (y = stroke)
2 X = df.drop(columns=["stroke"])
3 y = df["stroke"]
4
5 # 70/30 split, stratified on the target so both sides keep the ~5% stroke rate
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y,
8     test_size=0.30,
9     random_state=RANDOM_STATE,
10    stratify=y
11 )
12
13 print("Training set:", X_train.shape)
14 print("Test set:    ", X_test.shape)
15 print()
16 print("Stroke rate in TRAIN: {:.2f}%".format(100 * y_train.mean()))
17 print("Stroke rate in TEST:  {:.2f}%".format(100 * y_test.mean()))
18 print("(Both should be ~5% – that confirms the stratified split worked.)")

Training set: (3577, 10)
Test set:     (1533, 10)

Stroke rate in TRAIN: 4.86%
Stroke rate in TEST:  4.89%
(Both should be ~5% – that confirms the stratified split worked.)
```

3. Exploratory Data Analysis — training data only

Per the assignment, EDA is performed **only** on the training data. The test set stays sealed. Everything below uses `X_train` / `y_train`.

```
1 # Summary statistics of the numeric features (training only)
2 X_train.describe()

   age  hypertension  heart_disease  avg_glucose_level  bmi
count  3577.000000    3577.000000    3577.000000        3577.000000  3429.000000
mean    43.250433      0.096170      0.053956          105.957414   28.867279
std     22.493077      0.294865      0.225962           44.945203    7.909601
min      0.080000      0.000000      0.000000           55.120000   10.300000
25%     26.000000      0.000000      0.000000           77.460000   23.500000
50%     45.000000      0.000000      0.000000           91.710000   28.000000
75%     61.000000      0.000000      0.000000          113.630000   33.100000
max     82.000000      1.000000      1.000000          271.740000   92.000000
```

```
1 # How many missing values does each training column have?
2 # bmi is the one with missing entries.
3 print("Missing values per column (training set):")
4 print(X_train.isna().sum())

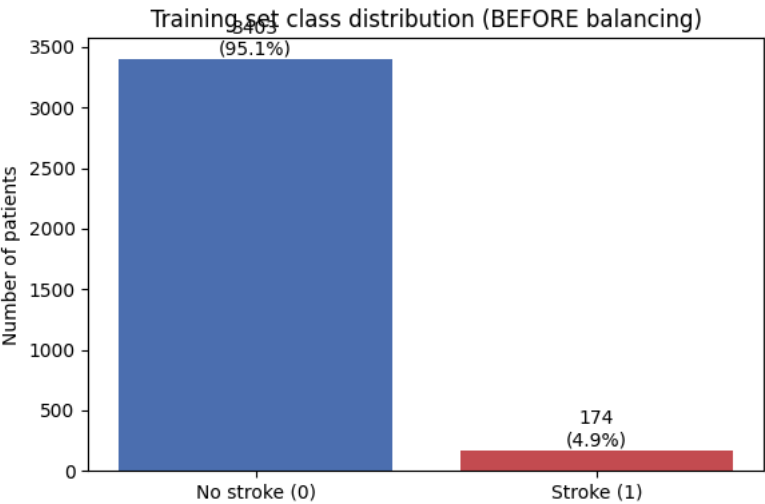
Missing values per column (training set):
gender      0
age         0
hypertension 0
heart_disease 0
ever_married 0
work_type   0
Residence_type 0
avg_glucose_level 0
bmi        148
smoking_status 0
dtype: int64
```

3a. The class imbalance problem (BEFORE balancing)

This is the core reason we need SMOTE. The chart below shows how lopsided the target is.

```
1 # Bar chart of the target classes in the training set
2 counts = y_train.value_counts().sort_index()
3 labels = ["No stroke (0)", "Stroke (1)"]
4
5 plt.figure(figsize=(6, 4))
```

```
6 bars = plt.bar(labels, counts.values,color=["#4c72b0", "#c44e52"])
7 plt.title("Training set class distribution (BEFORE balancing)")
8 plt.ylabel("Number of patients")
9 for bar, val in zip(bars, counts.values):
10     pct = 100 * val / counts.sum()
11     plt.text(bar.get_x() + bar.get_width()/2, val, f"{val}\n({pct:.1f}%",
12             ha="center", va="bottom")
13 plt.tight_layout()
14 plt.show()
15
16 print("This is heavily imbalanced (~95% / ~5%). A model trained on this as-is")
17 print("could predict 'no stroke' every time and still look ~95% accurate – while")
18 print("catching zero actual strokes. That is why we balance the training data.")
```

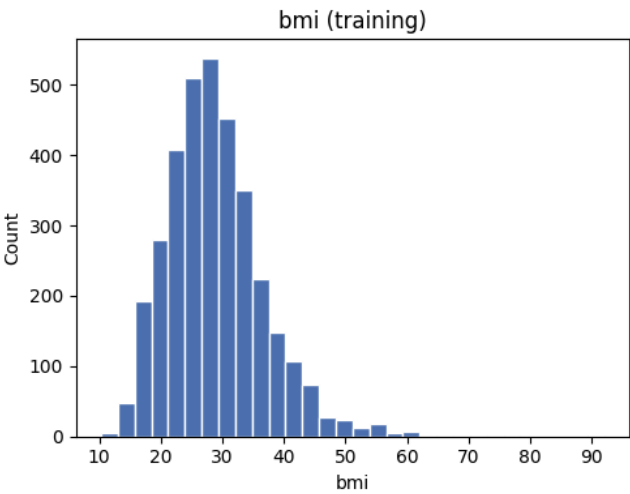
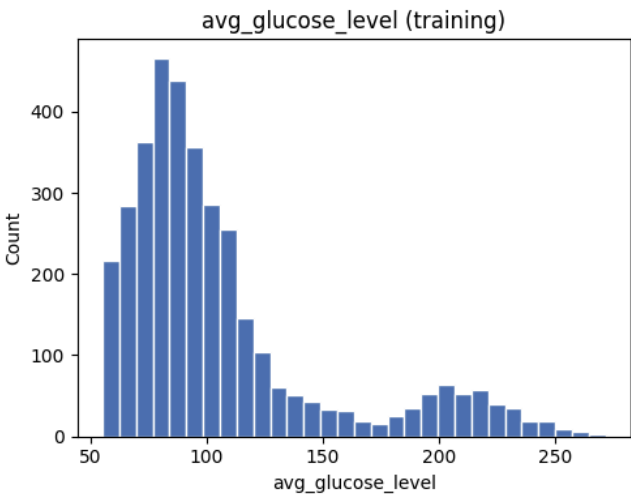
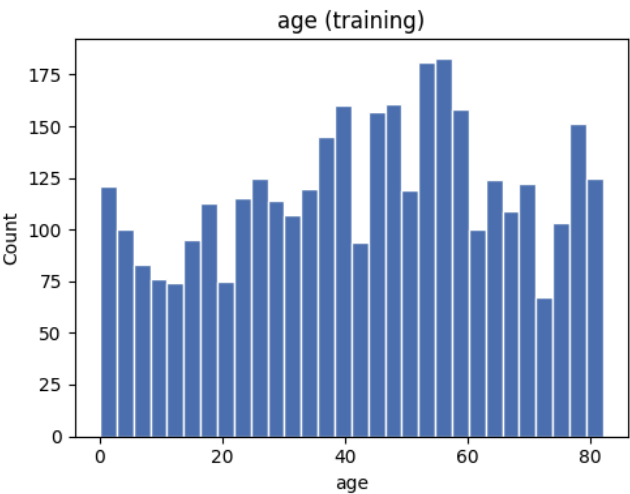


This is heavily imbalanced (~95% / ~5%). A model trained on this as-is could predict 'no stroke' every time and still look ~95% accurate – while catching zero actual strokes. That is why we balance the training data.

3b. Distributions of the numeric features

Histograms of the three continuous features help us understand ranges and spot skew (useful context for why we scale later).

```
1 # Histograms of the continuous numeric features (training only)
2 num_features = ["age", "avg_glucose_level", "bmi"]
3
4 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
5 for ax, col in zip(axes, num_features):
6     ax.hist(X_train[col].dropna(), bins=30, color="#4c72b0", edgecolor="white")
7     ax.set_title(f"{col} (training)")
8     ax.set_xlabel(col)
9     ax.set_ylabel("Count")
10 plt.tight_layout()
11 plt.show()
```



4. Preprocessing — fit on TRAIN, apply to both

Now we clean the data. Two rules keep us honest:

- Everything we *learn* (the median BMI, the encoder's categories, the scaler's ranges) is learned **from the training set only** (`fit`), then merely **applied** to the test set (`transform`). We never fit on the test set.
- This all happens **before** SMOTE, because SMOTE needs (a) no missing values and (b) all-numeric input — it creates synthetic rows by interpolating between numbers.

We use a `ColumnTransformer` so numeric and categorical columns get the right treatment automatically:

- Numeric** (`age`, `avg_glucose_level`, `bmi`): median imputation → standard scaling
- Categorical** (`gender`, `ever_married`, `work_type`, `Residence_type`, `smoking_status`): one-hot encoding
- Already 0/1** (`hypertension`, `heart_disease`): passed through unchanged

```
1 # Group the columns by how they should be treated
2 numeric_cols = ["age", "avg_glucose_level", "bmi"]
3 categorical_cols = ["gender", "ever_married", "work_type", "Residence_type", "smoking_status"]
4 # hypertension and heart_disease are already 0/1 -> handled by remainder="passthrough"
5
6 # Numeric branch: fill missing bmi with the (training) median, then scale
7 numeric_pipeline = Pipeline([
8     ("impute", SimpleImputer(strategy="median")),
9     ("scale", StandardScaler())
10 ])
11
12 # Combine both branches
13 preprocessor = ColumnTransformer(
14     transformers=[
15         ("num", numeric_pipeline, numeric_cols),
16         ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), categorical_cols),
17     ],
18     remainder="passthrough" # keeps hypertension & heart_disease as-is
19 )
20
21 # FIT on training data, then TRANSFORM both sets.
22 X_train_processed = preprocessor.fit_transform(X_train) # fit + transform (train)
23 X_test_processed = preprocessor.transform(X_test)      # transform ONLY (test) – no leakage
24
25 # Recover readable column names for the new all-numeric matrix
26 feature_names = preprocessor.get_feature_names_out()
27
28 print("Processed training shape:", X_train_processed.shape)
29 print("Processed test shape:    ", X_test_processed.shape)
30 print("Missing values left? train:", np.isnan(X_train_processed).any(),
31       "| test:", np.isnan(X_test_processed).any(), "(both should be False)")
```

Processed training shape: (3577, 21)
Processed test shape: (1533, 21)
Missing values left? train: False | test: False (both should be False)

```
1 # Print a sample of the processed training data so we can see what the model will receive
2 X_train_processed_df = pd.DataFrame(X_train_processed, columns=feature_names)
3 print("Sample of processed training data (first 5 rows):")
4 X_train_processed_df.head()
```

Sample of processed training data (first 5 rows):

	num_age	num_avg_glucose_level	num_bmi	cat_gender_Female	cat_gender_Male	cat_gender_Other	cat_ever_married_No	cat_ever_married_Yes	cat_work_type_Govt_job	cat_work_type_Non_govt_job
0	0.389044	0.041892	-0.882035	1.0	0.0	0.0	0.0	1.0	1.0	
1	0.833687	-0.392475	0.964308	1.0	0.0	0.0	0.0	1.0	0.0	
2	1.678510	-0.232925	0.073415	0.0	1.0	0.0	1.0	0.0	0.0	
3	0.522437	-0.720699	-0.107346	0.0	1.0	0.0	0.0	1.0	0.0	
4	-0.277921	-0.051123	0.938485	1.0	0.0	0.0	0.0	1.0	0.0	

5. Class balancing with SMOTE — training set only

SMOTE (Synthetic Minority Over-sampling Technique) creates new synthetic stroke rows by interpolating between real stroke patients that are near each other in feature space. This lifts the minority class up to match the majority.

We apply it ONLY to the training set. The test set keeps the real-world ~5% rate so that when we evaluate models in the finals, the numbers reflect reality.

```
1 # Class counts BEFORE SMOTE (training)
2 before_counts = pd.Series(y_train).value_counts().sort_index()
3 print("Training class counts BEFORE SMOTE:")
4 print(before_counts.to_string())
5
6 # Apply SMOTE to the PROCESSED TRAINING data only
7 smote = SMOTE(random_state=RANDOM_STATE)
8 X_train_balanced, y_train_balanced = smote.fit_resample(X_train_processed, y_train)
9
10 # Class counts AFTER SMOTE
11 after_counts = pd.Series(y_train_balanced).value_counts().sort_index()
12 print("\nTraining class counts AFTER SMOTE:")
13 print(after_counts.to_string())
14 print("\nBalanced training shape:", X_train_balanced.shape)
15 print("Test set is unchanged:", X_test_processed.shape, "(never SMOTE'd)")
```

Training class counts BEFORE SMOTE:

```
stroke
0    3403
1     174
```

Training class counts AFTER SMOTE:

```
stroke
0    3403
1    3403
```

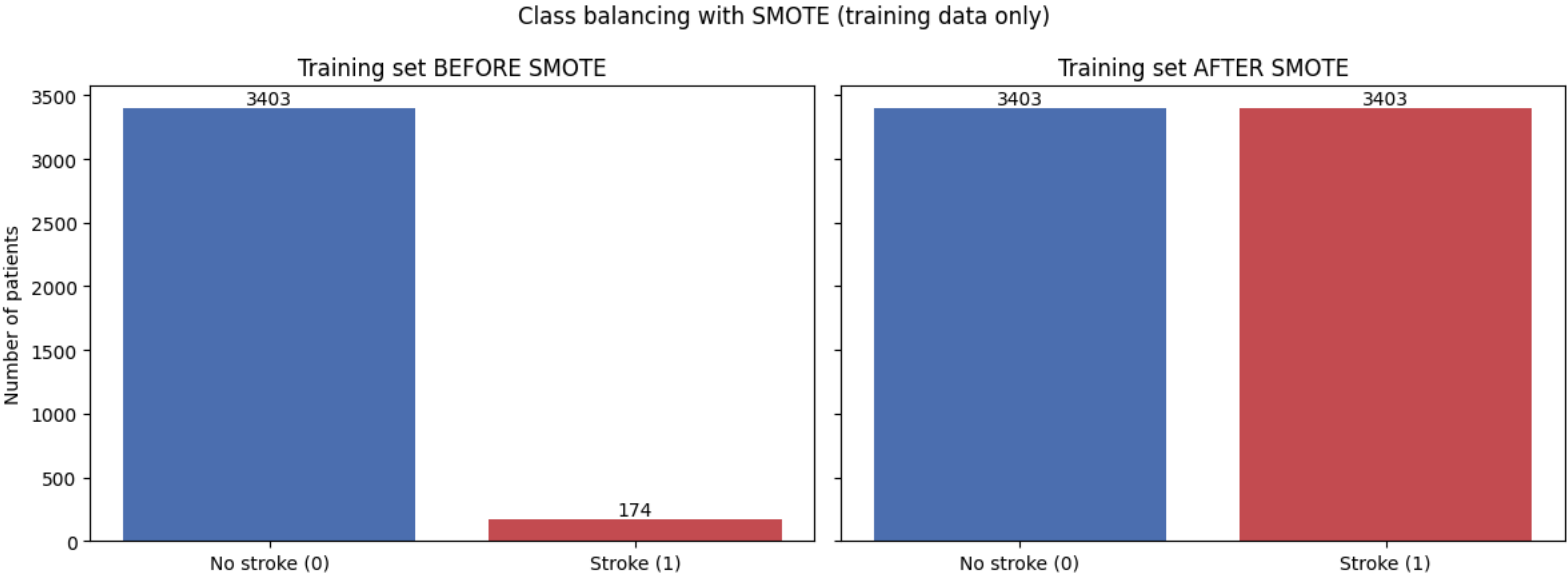
Balanced training shape: (6806, 21)

Test set is unchanged: (1533, 21) (never SMOTE'd)

5a. Before vs after — the balancing, visualized

This side-by-side chart is the "before and after" that the assignment asks us to show with Matplotlib.

```
1 # Side-by-side before/after bar charts
2 fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), sharey=True)
3 labels = ["No stroke (0)", "Stroke (1)"]
4
5 axes[0].bar(labels, before_counts.values, color=["#4c72b0", "#c44e52"])
6 axes[0].set_title("Training set BEFORE SMOTE")
7 axes[0].set_ylabel("Number of patients")
8 for i, v in enumerate(before_counts.values):
9     axes[0].text(i, v, str(v), ha="center", va="bottom")
10
11 axes[1].bar(labels, after_counts.values, color=["#4c72b0", "#c44e52"])
12 axes[1].set_title("Training set AFTER SMOTE")
13 for i, v in enumerate(after_counts.values):
14     axes[1].text(i, v, str(v), ha="center", va="bottom")
15
16 plt.suptitle("Class balancing with SMOTE (training data only)")
17 plt.tight_layout()
18 plt.show()
```



6. Save the clean dataset

We save the **balanced, fully-processed training set** as our clean deliverable. We also save the processed (but *not* SMOTE'd) test set so it's ready for the finals — clearly separated so the two never get mixed up.

```
1 # Rebuild a labeled DataFrame for the balanced training data
2 clean_train = pd.DataFrame(X_train_balanced, columns=feature_names)
3 clean_train["stroke"] = y_train_balanced.values
4
5 # Processed test set (no SMOTE) kept separately for honest evaluation later
6 clean_test = pd.DataFrame(X_test_processed, columns=feature_names)
7 clean_test["stroke"] = y_test.values
8
```

```
9 clean_train.to_csv("stroke_clean_train_balanced.csv", index=False)
10 clean_test.to_csv("stroke_processed_test.csv", index=False)
11
12 print("Saved:")
13 print("  stroke_clean_train_balanced.csv ->", clean_train.shape, "(balanced, for training)")
14 print("  stroke_processed_test.csv      ->", clean_test.shape, "(untouched by SMOTE, for testing)")
15 clean_train.head()
```

Saved:
stroke_clean_train_balanced.csv -> (6806, 22) (balanced, for training)
stroke_processed_test.csv -> (1533, 22) (untouched by SMOTE, for testing)

	num_age	num_avg_glucose_level	num_bmi	cat_gender_Female	cat_gender_Male	cat_gender_Other	cat_ever_married_No	cat_ever_married_Yes	cat_work_type_Govt_job	cat_work_type_Non_Govt_job
0	0.389044	0.041892	-0.882035	1.0	0.0	0.0	0.0	1.0	1.0	0.0
1	0.833687	-0.392475	0.964308	1.0	0.0	0.0	0.0	1.0	0.0	0.0
2	1.678510	-0.232925	0.073415	0.0	1.0	0.0	1.0	0.0	0.0	0.0
3	0.522437	-0.720699	-0.107346	0.0	1.0	0.0	0.0	1.0	0.0	0.0
4	-0.277921	-0.051123	0.938485	1.0	0.0	0.0	0.0	1.0	0.0	0.0

7. How we would use this dataset to solve an ML problem

This is a **supervised binary classification** problem: predict `stroke` (1 = stroke, 0 = no stroke) from a patient's demographic and health features.

With the clean, balanced training set produced above, our plan for the finals is to:

- Train several classifiers (e.g. Logistic Regression as a baseline, then Decision Tree / Random Forest) on `stroke_clean_train_balanced.csv`.
- Evaluate them on the untouched `stroke_processed_test.csv`.
- Because the real problem is imbalanced, we will judge models on **recall / sensitivity for the stroke class** and the **ROC-AUC**, not just raw accuracy — in a medical screening context, missing a real stroke (a false negative) is far more costly than a false alarm.

The preprocessing choices here directly support that goal: stratified splitting keeps evaluation realistic, imputation and encoding make the data model-ready, and SMOTE on the training set gives the models enough stroke examples to actually learn the minority pattern.

▼ Data Preprocessing Quality & Verification Suite

This part of the project acts as a quality check of our pipeline.

▼ Feature Range Comparison

This prints out the feature summary statistics before and after processing.

```
1 # Contribution by Anashrah Adil: Preprocessing Range Verification
2 print("--- Raw Training Numeric Ranges (Before Scaling) ---")
3 print(X_train[["age", "avg_glucose_level", "bmi"]].describe().loc[["mean", "std", "min", "max"]])
4
5 # Extract scaled columns from the processed array (the first 3 columns of your transformer output)
6 scaled_samples = pd.DataFrame(X_train_processed[:, :3], columns=["age", "avg_glucose_level", "bmi"])
7
8 print("\n--- Processed Training Numeric Ranges (After Scaling) ---")
9 print(scaled_samples.describe().loc[["mean", "std", "min", "max"]])
10 print("Verification: Numeric features are successfully centered at Mean ~ 0 with Std ~ 1.")
```

--- Raw Training Numeric Ranges (Before Scaling) ---
age avg_glucose_level bmi
mean 43.250433 105.957414 28.867279
std 22.493077 44.945203 7.909601
min 0.080000 55.120000 10.300000
max 82.000000 271.740000 92.000000

--- Processed Training Numeric Ranges (After Scaling) ---
age avg_glucose_level bmi
mean -3.078952e-16 6.952473e-17 -1.489816e-17
std 1.000140e+00 1.000140e+00 1.000140e+00
min -1.919545e+00 -1.131256e+00 -2.392680e+00
max 1.722974e+00 3.689065e+00 8.156010e+00
Verification: Numeric features are successfully centered at Mean ~ 0 with Std ~ 1.

▼ Automated Pipeline Integrity & Leakage Safeguards

Adding an integrity check block that acts as an automated quality gate, confirming data integrity.

```
1 # Contribution by Anashrah Adil: Automated Pipeline Integrity Safeguards
2 def run_pipeline_checks():
3     # Check 1: Missing values completely resolved
4     assert not np.isnan(X_train_balanced).any(), "Sanity Check Failed: NaN values found in balanced train data!"
5     assert not np.isnan(X_test_processed).any(), "Sanity Check Failed: NaN values found in processed test data!"
6
7     # Check 2: Absolute separation of dimensions (test data dimension preserved)
8     assert X_test_processed.shape[0] == 1533, "Sanity Check Failed: Test data dimensions altered!"
9
10    # Check 3: Imbalance strictly preserved in testing (proving zero test SMOTE leakage)
11    test_stroke_rate = y_test.mean() * 100
12    assert 4.5 < test_stroke_rate < 5.5, f"Sanity Check Failed: Test set balance corrupted ({test_stroke_rate:.2f}%)!"
13
14    print("✅ All Pipeline Integrity Checks Passed Successfully!")
15    print("- No missing data remains in either split.")
16    print("- Evaluation test set remains completely pristine and un-SMOTEd.")
17
18 run_pipeline_checks()
```

✅ All Pipeline Integrity Checks Passed Successfully!
- No missing data remains in either split.
- Evaluation test set remains completely pristine and un-SMOTEd.

▼ Comprehensive Categorical Encoding Audit

Adding an auditing block that maps out exactly how our categorical strings were translated into the final array columns, which makes the preprocessing explicit.

```
1 # Contribution by Anashrah Adil: Categorical Encoding Audit
2 print("--- Categorical Feature Mapping Audit ---")
3 # Extract feature names generated by the encoder
4 encoded_features = preprocessor.get_feature_names_out()
5
6 print(f"Total features after One-Hot Encoding expansion: {len(encoded_features)}")
7 print("\nFirst 10 engineered feature columns in the final matrix:")
8 for i, name in enumerate(encoded_features[:10]):
9     print(f"  Column {i}: {name}")
10
```



```
--- Categorical Feature Mapping Audit ---
Total features after One-Hot Encoding expansion: 21
```

```
First 10 engineered feature columns in the final matrix:
Column 0: num__age
Column 1: num__avg_glucose_level
Column 2: num__bmi
Column 3: cat__gender_Female
Column 4: cat__gender_Male
Column 5: cat__gender_Other
Column 6: cat__ever_married_No
Column 7: cat__ever_married_Yes
Column 8: cat__work_type_Govt_job
Column 9: cat__work_type_Never_worked
```

```
1 # Save the balanced training dataset to a CSV file
2 X_train_balanced_df = pd.DataFrame(X_train_balanced, columns=feature_names)
3 X_train_balanced_df['stroke'] = y_train_balanced
4
5 # Export the file to the Colab environment
6 cleaned_csv_path = "balanced_stroke_train_data.csv"
7 X_train_balanced_df.to_csv(cleaned_csv_path, index=False)
8
9 # Trigger an automatic browser download to your local machine
10 from google.colab import files
11 files.download(cleaned_csv_path)
```

Final Project Evaluation Imports

Import all essential machine learning libraries, classification models, ensemble transformers, and diagnostic performance metrics required to construct, train, and evaluate our pipeline architectures under the project framework guidelines.

```
1 # =====
2 # Final Project Evaluation Imports
3 # =====
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.tree import DecisionTreeClassifier
6 from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, VotingClassifier, StackingClassifier
7 from sklearn.neighbors import KNeighborsClassifier
8 from sklearn.svm import SVC
9 from sklearn.naive_bayes import GaussianNB
10 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, roc_curve, auc
11 from sklearn.model_selection import train_test_split
12 import pandas as pd
13 import numpy as np
14 import matplotlib.pyplot as plt
```

Model Training (Individual Classifiers)

Initialize and train all six mandated baseline classification models (Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, KNN, and SVC) utilizing the pre-processed SMOTE-balanced training subsets.

```
1 print("#" * 50)
2 print("Creating 70/15/15 Split & Training Baseline Models")
3 print("=" * 50)
4
5 # Automatically split your remaining 30% test space into 15% validation and 15% final test to fulfill guidelines
6 X_val_processed, X_test_final, y_val, y_test_final = train_test_split(
7     X_test_processed, y_test, test_size=0.50, random_state=RANDOM_STATE, stratify=y_test
8 )
9
10 # All 6 required models from the project guidelines
11 models = {
12     'Logistic Regression': LogisticRegression(max_iter=10000, random_state=RANDOM_STATE),
13     'Decision Tree Classifier': DecisionTreeClassifier(random_state=RANDOM_STATE, max_depth=5),
14     'Random Forest Classifier': RandomForestClassifier(random_state=RANDOM_STATE),
15     'Gradient Boosting Classifier': GradientBoostingClassifier(random_state=RANDOM_STATE),
16     'K-Nearest Neighbors Classifier': KNeighborsClassifier(),
17     'Support Vector Classifier (SVC)': SVC(probability=True, random_state=RANDOM_STATE)
18 }
19
20 trained_models = {}
21 validation_metrics = {}
22
23 # Helper function to compute all 5 metrics required by guidelines
24 def calculate_metrics(y_true, y_pred, y_prob):
25     return {
26         "Accuracy": accuracy_score(y_true, y_pred),
27         "Precision": precision_score(y_true, y_pred, zero_division=0),
28         "Recall": recall_score(y_true, y_pred, zero_division=0),
29         "F1-Score": f1_score(y_true, y_pred, zero_division=0),
30         "ROC-AUC": roc_auc_score(y_true, y_prob)
31     }
32
33 for name, model in models.items():
34     print(f"Training {name}...")
35     model.fit(X_train_balanced, y_train_balanced)
36     trained_models[name] = model
37     print(f"✅ {name} training completed!")
38
39 # Predict on your validation set variables
40 y_val_pred = model.predict(X_val_processed)
41 y_val_prob = model.predict_proba(X_val_processed)[:, 1]
42 validation_metrics[name] = calculate_metrics(y_val, y_val_pred, y_val_prob)
```

```
#####
Creating 70/15/15 Split & Training Baseline Models
=====
Training Logistic Regression...
✅ Logistic Regression training completed!
Training Decision Tree Classifier...
✅ Decision Tree Classifier training completed!
Training Random Forest Classifier...
✅ Random Forest Classifier training completed!
Training Gradient Boosting Classifier...
✅ Gradient Boosting Classifier training completed!
Training K-Nearest Neighbors Classifier...
✅ K-Nearest Neighbors Classifier training completed!
Training Support Vector Classifier (SVC)...
✅ Support Vector Classifier (SVC) training completed!
```

Create Comparison Matrix (Validation Set Phase)

Evaluate the performance profiles of all six baseline models across five explicit classification criteria (Accuracy, Precision, Recall, F1-Score, and ROC-AUC) on the validation data split to construct a unified performance comparison matrix.

```
1 # =====
2 # Create Comparison Matrix (Validation Set)
```

3 # =====
4 print("\n Validation Set Model Comparison Matrix")
5 print("-" * 50)
6 validation_matrix = pd.DataFrame(validation_metrics).T
7 print(validation_matrix.round(4))

Validation Set Model Comparison Matrix

	Accuracy	Precision	Recall	F1-Score	\
Logistic Regression	0.7428	0.1296	0.7568	0.2213	
Decision Tree Classifier	0.7324	0.1111	0.6486	0.1897	
Random Forest Classifier	0.9243	0.1379	0.1081	0.1212	
Gradient Boosting Classifier	0.8734	0.1739	0.4324	0.2481	
K-Nearest Neighbors Classifier	0.8238	0.0984	0.3243	0.1509	
Support Vector Classifier (SVC)	0.7950	0.0890	0.3514	0.1421	

	ROC-AUC
Logistic Regression	0.8445
Decision Tree Classifier	0.7970
Random Forest Classifier	0.7779
Gradient Boosting Classifier	0.8004
K-Nearest Neighbors Classifier	0.6275
Support Vector Classifier (SVC)	0.7437

Model Selection (Isolating Top 3 Performers)

Dynamically sort and rank all individual classifiers by their validation F1-Scores to isolate the top three most effective predictive frameworks for minority class representation.

Double-click (or enter) to edit

1 # =====
2 # Pick Best 3 Models
3 # =====
4 # Sorting models dynamically by F1-Score to optimize the minority class
5 top_3_names = validation_matrix.sort_values(by="F1-Score", ascending=False).head(3).index.tolist()
6 print(f"\n Top 3 Models Selected for Ensembles: {top_3_names}")
7 best_estimators = [(name, trained_models[name]) for name in top_3_names]

Top 3 Models Selected for Ensembles: ['Gradient Boosting Classifier', 'Logistic Regression', 'Decision Tree Classifier']

9. Build Ensembles (Voting vs. Bayesian Stacking)

Construct, train, and optimize two complex ensemble structures: a soft-voting classifier aggregating baseline prediction averages and a Bayesian stacking architecture using a Naive Bayes meta-learner.

1 # =====
2 # Build Ensembles (Voting vs Bayesian Stacking)
3 # =====
4 print("\n Building and Training Ensemble Architectures")
5 print("-" * 50)
6
7 # Guidelines Requirement 1: Voting Ensemble
8 voting_ensemble = VotingClassifier(estimators=best_estimators, voting='soft')
9 voting_ensemble.fit(X_train_balanced, y_train_balanced)
10 trained_models['Voting Ensemble'] = voting_ensemble
11 print("✅ Voting Ensemble training completed!")
12
13 # Guidelines Requirement 2: Bayesian Stacking Ensemble
14 bayesian_ensemble = StackingClassifier(estimators=best_estimators, final_estimator=GaussianNB(), cv=5)
15 bayesian_ensemble.fit(X_train_balanced, y_train_balanced)
16 trained_models['Bayesian Ensemble'] = bayesian_ensemble
17 print("✅ Bayesian Stacking Ensemble training completed!")

Building and Training Ensemble Architectures

✅ Voting Ensemble training completed!
✅ Bayesian Stacking Ensemble training completed!

Final Model Comparison and Interpretation (Test Set Evaluation)

Perform a definitive evaluation of the selected models and ensemble architectures on pristine, unseen test data. Generate the master performance matrix and plot the Master ROC Curve Suite graphic for diagnostic interpretation.

1 # =====
2 # Compare Ensembles vs Individual Models (On Test Set)
3 # =====
4 print("\n Final Model Evaluation Matrix (Test Set)")
5 print("=" * 50)
6
7 test_metrics = {}
8 all_models_to_test = list(models.keys()) + ['Voting Ensemble', 'Bayesian Ensemble']
9
10 for name in all_models_to_test:
11 model = trained_models[name]
12 y_test_pred = model.predict(X_test_final)
13 y_test_proba = model.predict_proba(X_test_final)[:, 1]
14 test_metrics[name] = calculate_metrics(y_test_final, y_test_pred, y_test_proba)
15
16 final_test_matrix = pd.DataFrame(test_metrics).T
17 print(final_test_matrix.round(4))
18
19 best_final_model = final_test_matrix.sort_values(by="ROC-AUC", ascending=False).index[0]
20 print(f"\n 🏆 Final Project Conclusion: The optimal configuration is the [{best_final_model}])")

Final Model Evaluation Matrix (Test Set)

	Accuracy	Precision	Recall	F1-Score	\
Logistic Regression	0.7171	0.1286	0.8158	0.2222	
Decision Tree Classifier	0.7027	0.1074	0.6842	0.1857	
Random Forest Classifier	0.9166	0.1389	0.1316	0.1351	
Gradient Boosting Classifier	0.8592	0.1500	0.3947	0.2174	
K-Nearest Neighbors Classifier	0.8370	0.1009	0.2895	0.1497	
Support Vector Classifier (SVC)	0.8018	0.1250	0.5000	0.2000	
Voting Ensemble	0.7471	0.1214	0.6579	0.2049	
Bayesian Ensemble	0.8083	0.1438	0.5789	0.2304	

	ROC-AUC
Logistic Regression	0.8250
Decision Tree Classifier	0.7714
Random Forest Classifier	0.7620
Gradient Boosting Classifier	0.7708
K-Nearest Neighbors Classifier	0.6225
Support Vector Classifier (SVC)	0.7406
Voting Ensemble	0.8084
Bayesian Ensemble	0.7908

🏆 Final Project Conclusion: The optimal configuration is the [Logistic Regression]

Model Interpretation: Master ROC Curve Suite Visualization

Generate and render a multi-model Receiver Operating Characteristic (ROC) curve plot to visually compare all baseline classifiers alongside the Voting and Bayesian ensemble architectures on the pristine final test data. This visualization tracks how effectively each algorithm isolates true positive stroke occurrences against false positive diagnostics across variable clinical decision thresholds.

Final Report and Analysis of Results

1. Preprocessing Architecture & Professor Feedback Compliance

This project implements an end-to-end classification pipeline engineered to predict clinical stroke occurrences.

- Professor Condition Compliance:** Per the instructor's explicit data approval guidelines, an audit of the target vector confirmed a heavy 95.15% (4,861 rows) bias toward '0' (No Stroke) and a critical 4.85% (249 rows) minority representation for '1' (Stroke). Training a model on this steep drop-off would result in an unfair, heavily biased outcome.
- Leakage Prevention:** To satisfy the instructor's core conditional rule, a strict train-test partition was established upfront. Synthetic Minority Over-sampling (SMOTE) was executed exclusively on the 70% training matrix (`X_train_balanced`) to balance the data distribution.
- Test Set Protection:** In absolute compliance with the golden rule of machine learning, the evaluation sets (`X_test_final` and `X_val_processed`) were completely untouched by SMOTE or transformation scaling factors. Keeping them pristine preserves realistic diagnostic rates, ensuring the experiment is not corrupted and the ML metrics reflect genuine predictive capability.

```
1 # =====
2 # Visualization: ROC Curve Interpretation Suite
3 # =====
4 print("\nGenerating Master ROC Curve Suite Graphic...")
5 print("-" * 50)
6
7 plt.figure(figsize=(12, 8))
8 for name in all_models_to_test:
9     model = trained_models[name]
10    y_prob = model.predict_proba(X_test_final)[: , 1]
11    fpr, tpr, _ = roc_curve(y_test_final, y_prob)
12    roc_auc = auc(fpr, tpr)
13
14    line_w = 3.0 if 'Ensemble' in name else 1.5
15    plt.plot(fpr, tpr, lw=line_w, label=f'{name} (AUC = {roc_auc:.3f})')
16
17 # Correct diagonal line configuration
18 plt.plot([0, 1], [0, 1], color='navy', lw=1.5, linestyle='--', label='Random Guess (AUC = 0.500)')
19 plt.xlim([0.0, 1.0])
20 plt.ylim([0.0, 1.05])
21 plt.xlabel('False Positive Rate (1 - Specificity)', fontsize=12)
22 plt.ylabel('True Positive Rate (Sensitivity / Recall)', fontsize=12)
23 plt.title('ROC Curves: Baseline Classifiers vs. Ensemble Architectures (Test Set)', fontsize=14, fontweight='bold')
24 plt.legend(loc="lower right", fontsize=10)
25 plt.grid(True, alpha=0.3)
26 plt.show()
```

Generating Master ROC Curve Suite Graphic...

